

★ Get unlimited access to the best of Medium for less than \$1/week. [Become a member](#)



展開數據人生

Home

About

【Python 機器學習】110：簡單貝氏分類器介紹及其應用

Naive Bayes Classifier



張育晟 Eason Chang · [Follow](#)

Published in 展開數據人生 · Sep 13, 2020



975





接下來要介紹的分類器叫簡單(單純)貝氏分類器，那什麼是貝氏呢？

貝氏定理是機率中的一個定理，**描述在已知一些條件下，某事件的發生機率**。通常，事件 A 在事件 B 已發生的條件下發生的機率，與事件 B 在事件 A 已發生的條件下發生的機率是不一樣的。

$$P(A|B) = \frac{P(B|A) P(A)}{P(B)}$$

貝氏定理的公式

那貝氏定理又和分類器有什麼關聯呢？首先我們要先知道分類器模型其實是把連續型的輸出 Xw 轉換到離散型的輸出 $\hat{y}$ ，中間會經過兩個函數的運算，第一個函數用 σ ，第二個則用 H 表示。

$$\hat{y} = g(h(X)) = H(\sigma(Xw))$$

其中 σ 俗稱為激勵函數 (Activation function)， H 稱為階躍函數 (Step function)

激勵函數將連續型輸出轉換為機率(0~1)

階躍函數將機率輸出轉換為離散值

而貝氏分類器只在意第一個函數(機率的計算)，也就是說，該分類器的輸出可以用機率表示：給定 x 則其類別為 C_i 的機率為多少

$$\sigma(x^T w) = P(C_i | x)$$

其中， $P(C_i | x)$ 即我們俗稱的後驗機率(Posterior Probability)

我們一樣使用鐵達尼號的範例，使用簡單貝氏分類器分類乘客的生存與否：

```
1 import pandas as pd
2
3 titanic = pd.read_csv('http://storage.googleapis.com/kaggle_datasets/Titanic-Machine-Learning-from
4 titanic_subset = titanic[['Pclass', 'Survived']]
5 titanic_subset.head()
```



nvml11001 nv hosted with ❤️ by GitHub

[view raw](#)

Out [2]:

	Pclass	Survived
0	3	0
1	1	1
2	3	1
3	1	1
4	3	0

先使用 `pd.crosstab()` 製作 Frequency Table，以方便我們計算機率：

```
1 freq_table = pd.crosstab(index = titanic_subset['Survived'], columns = titanic_subset['Pclass'])
2 freq_table
```

pym11002.py hosted with ❤ by GitHub

[view raw](#)

Out [3]:

Pclass	1	2	3
Survived			
0	80	97	372
1	136	87	119

我們將乘客分割成 6 個類別(Pclass3 個，Survived2 個)，用這 6 個區塊的數字就可以分別計算後驗機率：

$$P(\text{Survived} | \text{Pclass} = 1) = \frac{P(\text{Pclass} = 1 | \text{Survived}) \times P(\text{Survived})}{P(\text{Pclass} = 1)}$$
$$P(\text{Survived} | \text{Pclass} = 2) = \frac{P(\text{Pclass} = 2 | \text{Survived}) \times P(\text{Survived})}{P(\text{Pclass} = 2)}$$
$$P(\text{Survived} | \text{Pclass} = 3) = \frac{P(\text{Pclass} = 3 | \text{Survived}) \times P(\text{Survived})}{P(\text{Pclass} = 3)}$$

左邊的機率是我們屆時分類的依據，當一個 $\text{Pclass}=1$ 的樣本進來時，究竟要猜他是存活還是死亡，就是根據計算得到的後驗機率，如果進來的 $\text{Pclass}=2$ ，就看 Pclass 為 2 的後驗機率，以此類推。

為方便計算，先把所需要的數字都先準備好：

```
1 num_passengers = freq_table.sum().sum()
2 num_survived = freq_table.loc[1, :].sum()
3 num_pclass1 = freq_table.loc[:, 1].sum()
4 num_pclass2 = freq_table.loc[:, 2].sum()
5 num_pclass3 = freq_table.loc[:, 3].sum()
6
7 num_pclass1_survived = freq_table.loc[:, 1][1]
8 num_pclass2_survived = freq_table.loc[:, 2][1]
9 num_pclass3_survived = freq_table.loc[:, 3][1]
```

pyml11003.py hosted with ❤ by GitHub

[view raw](#)

注意：以上程式碼使用 loc 函數，括號裡放的是標籤的名字而不是索引值，所以程式碼中的 123 是標籤名字，而不是相對位置

分別去計算分母以及分子的機率：

```
1  p_pclass1 = num_pclass1 / num_passengers
2  p_pclass2 = num_pclass2 / num_passengers
3  p_pclass3 = num_pclass3 / num_passengers
4
5  p_survived = num_survived / num_passengers
6
7  p_class1_survived = num_pclass1_survived / num_survived
8  p_class2_survived = num_pclass2_survived / num_survived
9  p_class3_survived = num_pclass3_survived / num_survived
```

pym11004.py hosted with ❤ by GitHub

[view raw](#)

計算 3 個 Pclass 的後驗機率：

```
1  p_survived_pclass1 = p_class1_survived * p_survived / p_pclass1
2  p_survived_pclass2 = p_class2_survived * p_survived / p_pclass2
3  p_survived_pclass3 = p_class3_survived * p_survived / p_pclass3
```

pym11005.py hosted with ❤ by GitHub

[view raw](#)

```
1 print(p_survived_pclass1)
2 print(p_survived_pclass2)
3 print(p_survived_pclass3)
```

pym11006.py hosted with ❤ by GitHub

[view raw](#)

```
0.6296296296296295
0.47282608695652173
0.24236252545824846
```

如上圖，3個後驗機率分別求出來了後，假如現在得到一個樣本，他的 Pclass=1，那我就有63%的機率去猜該乘客是存活的，相反，有37%的機率猜他是死的。果然與上一章提到的專家意見的建議差不多，倉等越高的顧客存活率越高，make sense！！

我們現在算出來的機率並不是 training data 的機率，只是把整個 data 拿來算

如何操作簡單貝氏分類器來預測乘客的生存與否：

```

1  import pandas as pd
2  from sklearn.model_selection import train_test_split
3
4  titanic = pd.read_csv('http://storage.googleapis.com/kaggle_datasets/Titanic-Machine-Learning-frc
5  X = titanic['Pclass'].values.reshape(-1, 1)
6  y = titanic['Survived'].values
7  X_train, X_validation, y_train, y_validation = train_test_split(X, y, test_size = 0.33, random_st
8  train_df = pd.DataFrame()
9  train_df['Survived'] = y_train
10 train_df['Pclass'] = X_train.ravel()

```

nvml11007 nv hosted with ❤ by GitHub

[view raw](#)

注意：傳資料回資料框時，記得先使用 `ravel()` 函數轉回一維陣列

製作訓練資料的 Frequency Table：

```

1  freq_table = pd.crosstab(index = train_df['Survived'], columns = train_df['Pclass'])
2  freq_table

```

pym11008.py hosted with ❤ by GitHub

[view raw](#)

Out[29]:

Pclass	1	2	3
Survived			
0	55	64	255
1	80	61	81

如此一來，training data 就會跟先前的 titanic subset 差不多，只是樣本數較少

計算訓練資料的後驗機率：

```
1  num_passengers = freq_table.sum().sum()
2  num_survived = freq_table.loc[1, :].sum()
3  num_pclass1 = freq_table.loc[:, 1].sum()
4  num_pclass2 = freq_table.loc[:, 2].sum()
5  num_pclass3 = freq_table.loc[:, 3].sum()
6  num_pclass1_survived = freq_table.loc[:, 1][1]
7  num_pclass2_survived = freq_table.loc[:, 2][1]
8  num_pclass3_survived = freq_table.loc[:, 3][1]
9
10 p_pclass1 = num_pclass1 / num_passengers
11 p_pclass2 = num_pclass2 / num_passengers
12 p_pclass3 = num_pclass3 / num_passengers
13 p_survived = num_survived / num_passengers
14 p_class1_survived = num_pclass1_survived / num_survived
15 p_class2_survived = num_pclass2_survived / num_survived
16 p_class3_survived = num_pclass3_survived / num_survived
17
18 p_survived_pclass1 = p_class1_survived * p_survived / p_pclass1
19 p_survived_pclass2 = p_class2_survived * p_survived / p_pclass2
20 p_survived_pclass3 = p_class3_survived * p_survived / p_pclass3
21
22 print(p_survived_pclass1)
23 print(p_survived_pclass2)
24 print(p_survived_pclass3)
```

pym11009.py hosted with ❤ by GitHub

[view raw](#)

0.5925925925925926
0.4880000000000001
0.24107142857142858

如上圖，我們一樣輕鬆的把後驗機率算出來，雖然值和剛才略有不一樣，但還是和專家意見預期的差不多，越高倉等的就要用越高的機率去猜他存活。但是在有了後驗機率之後，到底要怎麼把它應用到我們的 `X_validation` 上面來預測 `y_validation` 呢？

將後驗機率應用至 `X_validation`，得到 $\hat{y}$ ：

```
1 import numpy as np
2
3 def get_y_pred(X_validation, p_survived_pclass1, p_survived_pclass2, p_survived_pclass3):
4     y_pred = np.zeros(X_validation.shape[0], dtype = int)
5     for i in range(y_pred.size):
6         if X_validation[i, 0] == 1:
7             prediction = np.random.choice([0, 1], p = [1 - p_survived_pclass1, p_survived_pclass1])
8         elif X_validation[i, 0] == 2:
9             prediction = np.random.choice([0, 1], p = [1 - p_survived_pclass2, p_survived_pclass2])
10        elif X_validation[i, 0] == 3:
11            prediction = np.random.choice([0, 1], p = [1 - p_survived_pclass3, p_survived_pclass3])
12        y_pred[i] = prediction
13
14    return y_pred
```



我們使用 `choice` 函數，給一個陣列(0, 1)從裡面隨機挑選，之前在不給 `p` 參數的情況下就是各 50%。現在我們把後驗機率放進去：假如它的 `Pclass` 為 1，則預測該乘客死亡(0)的機率為(1 - 0.24)，存活(1)的機率為(0.24)

```
1 y_pred = get_y_pred(X_validation, p_survived_pclass1, p_survived_pclass2, p_survived_pclass3)
2 print(y_pred[:10])
3 print(y_validation[:10])
```

pym11011.py hosted with ❤ by GitHub

[view raw](#)

```
[0 1 0 1 1 1 0 1 0 0]
[1 0 0 1 1 1 1 0 1 1]
```

我們印出前十筆預測結果和正確答案(`y_validation`)做比較，假如上下是一樣的，就表示我們的預測是對的，假如不一樣，那預測就是錯的。

```
1 n_tp_tn = (y_pred == y_validation).sum()
2 acc = n_tp_tn / y_pred.size
3 print('Accuracy score: {:.2f}%'.format(acc * 100))
```

pym11012.py hosted with ❤ by GitHub

[view raw](#)**Accuracy score: 56.95%**

使用準確率檢查該分類器的表現

如果簡單貝氏分類器希望納入多個類別變數呢？

在專家意見中，除了艙等以外，性別也是影響生存率的，女性較男性易生存。如果要把性別加入該分類模型，應該要如何操作呢？

一樣先製作訓練資料的 Frequency Table:

```
1 import pandas as pd
2 from sklearn.model_selection import train_test_split
3
4 titanic = pd.read_csv('http://storage.googleapis.com/kaggle_datasets/Titanic-Machine-Learning-from
5 X = titanic[['Pclass', 'Sex']].values
6 y = titanic['Survived'].values
7 X_train, X_validation, y_train, y_validation = train_test_split(X, y, test_size = 0.33, random_sta
```

nvml11013 nv hosted with ❤️ by GitHub

[view raw](#)

```
1 train_df = pd.DataFrame()
2 train_df['Pclass'] = X_train[:, 0].ravel()
3 train_df['Sex'] = X_train[:, 1].ravel()
4 train_df['Survived'] = y_train
5 freq_table = pd.crosstab(index = train_df['Survived'], columns = [train_df['Pclass'], train_df['Se
6 freq_table
```

nvml11014 nv hosted with ❤️ by GitHub

[view raw](#)

Out [46]:

Pclass	1		2		3	
Sex	female	male	female	male	female	male
Survived						
0	3	52	1	63	53	202
1	56	24	47	14	46	35

如上圖，如果再加入一個類別變數，則 Frequency Table 外觀就會變得比較麻煩，且其索引會成一個巢狀外觀，每一個 Pclass 對應到兩種性別，所以要抓的樣本個數就會變得更多，計算也會更多，最後才會得到 6 個後驗機率。

計算有 n 個特徵的後驗機率(泛化形式)：

$$P(C_i|x_0, x_1, \dots, x_n) \propto P(x_0, x_1, \dots, x_n|C_i)P(C_i) \propto P(C_i) \prod_{j=1}^n P(x_j|C_i)$$

在泛化的公式中有個假設：特徵和特徵之間是獨立的，互相不影響，為什麼要有這個假設呢？這樣它的機率才可以互相相乘，而這也是為什麼該分類器叫“簡單”貝氏分類器的原因。

但在現實中，如果該模型含有多個類別變數，則特徵之間一般是不會互相獨立的。舉鐵達尼號的例子，Pclass 通常跟年紀有關，年紀越大，會有比較大的機

率坐到頭等艙。

計算訓練資料的後驗機率：

```
1  n_passengers = freq_table.sum().sum()
2  n_survived = freq_table.loc[1, :].sum()
3  n_Pclass1_survived = freq_table.loc[1, 1].sum()
4  n_Pclass2_survived = freq_table.loc[1, 2].sum()
5  n_Pclass3_survived = freq_table.loc[1, 3].sum()
6  n_female_survived = freq_table.loc[1, :].loc[:, 'female'].sum()
7  n_male_survived = freq_table.loc[1, :].loc[:, 'male'].sum()
```

pym11015.py hosted with ❤ by GitHub

[view raw](#)

```
1  p_survived = n_survived / n_passengers
2  p_Pclass1_survived = n_Pclass1_survived / n_survived
3  p_Pclass2_survived = n_Pclass2_survived / n_survived
4  p_Pclass3_survived = n_Pclass3_survived / n_survived
5  p_female_survived = n_female_survived / n_survived
6  p_male_survived = n_male_survived / n_survived
```

pym11016.py hosted with ❤ by GitHub

[view raw](#)

```
1 p_survived_Pclass1_female = p_survived * p_Pclass1_survived * p_female_survived
2 p_survived_Pclass1_male = p_survived * p_Pclass1_survived * p_male_survived
3 p_survived_Pclass2_female = p_survived * p_Pclass2_survived * p_female_survived
4 p_survived_Pclass2_male = p_survived * p_Pclass2_survived * p_male_survived
5 p_survived_Pclass3_female = p_survived * p_Pclass3_survived * p_female_survived
6 p_survived_Pclass3_male = p_survived * p_Pclass3_survived * p_male_survived
```

pym11016.py hosted with ❤ by GitHub

[view raw](#)

```
1 print(p_survived_Pclass1_female)
2 print(p_survived_Pclass1_male)
3 print(p_survived_Pclass2_female)
4 print(p_survived_Pclass2_male)
5 print(p_survived_Pclass3_female)
6 print(p_survived_Pclass3_male)
```

pym11017.py hosted with ❤ by GitHub

[view raw](#)

```
0.0900900900900901
0.044138097829373
0.06869369369369371
0.033655299594896916
0.09121621621621621
0.04468982405224016
```

有了這 6 個後驗機率之後，就可以去判斷 validation 的資料。將後驗機率應用至 X_validation，得到 $\hat{y}$ ：

```

1  import numpy as np
2
3  def get_y_pred(X, p_survived_Pclass1_female, p_survived_Pclass1_male, p_survived_Pclass2_female,
4                n_obs = X.shape[0])
5      y_pred = np.zeros(n_obs, dtype = int)
6      for i in range(n_obs):
7          if X[i, 0] == 1 and X[i, 1] == 'female':
8              y_pred[i] = np.random.choice([0, 1], p = [1 - p_survived_Pclass1_female, p_survived_Pclass1_female])
9          if X[i, 0] == 1 and X[i, 1] == 'male':
10             y_pred[i] = np.random.choice([0, 1], p = [1 - p_survived_Pclass1_male, p_survived_Pclass1_male])
11          if X[i, 0] == 2 and X[i, 1] == 'female':
12             y_pred[i] = np.random.choice([0, 1], p = [1 - p_survived_Pclass2_female, p_survived_Pclass2_female])
13          if X[i, 0] == 2 and X[i, 1] == 'male':
14             y_pred[i] = np.random.choice([0, 1], p = [1 - p_survived_Pclass2_male, p_survived_Pclass2_male])
15          if X[i, 0] == 3 and X[i, 1] == 'female':
16             y_pred[i] = np.random.choice([0, 1], p = [1 - p_survived_Pclass3_female, p_survived_Pclass3_female])
17          if X[i, 0] == 3 and X[i, 1] == 'male':
18             y_pred[i] = np.random.choice([0, 1], p = [1 - p_survived_Pclass3_male, p_survived_Pclass3_male])
19      return y_pred

```

nvml11018 nv hosted with ❤️ by GitHub

[view raw](#)

```

1  y_pred = get_y_pred(X_validation, p_survived_Pclass1_female, p_survived_Pclass1_male, p_survived_Pclass2_female,
2                    n_obs = X_validation.shape[0])
3  n_tp_tn = (y_pred == y_validation).sum()
4  acc = n_tp_tn / y_validation.size
5  print('Accuracy score: {:.2f}%'.format(acc * 100))

```

nvml11019 nv hosted with ❤️ by GitHub

[view raw](#)

Accuracy score: 60.34%

那如果簡單貝氏分類器希望納入數值變數呢？（**ex**：年齡）

```
1 import pandas as pd
2 from sklearn.model_selection import train_test_split
3
4 titanic = pd.read_csv('http://storage.googleapis.com/kaggle_datasets/Titanic-Machine-Learning-from
5 titanic = titanic[titanic['Age'].notnull()]
6 X = titanic['Age'].values.reshape(-1, 1)
7 y = titanic['Survived'].values
8 X_train, X_validation, y_train, y_validation = train_test_split(X, y, test_size = 0.33, random_sta
```



nvml11020 nv hosted with ❤️ by GitHub

[view raw](#)

回顧一下貝氏定理的公式，會發現根本應用不了數值變數，光是創一個 Frequency Table 檯面就會炸掉了。那應該要怎麼辦才好呢？

假如你碰到數值變數，但還是堅持使用簡單貝氏分類器的話，常用方法有兩種：

將數值變數分組為類別變數(能有效的將outlier剔除)

定義分組規則並向量化映射(可採用 **map**, **apply** 或 **np.vectorize()**)

使用 `pd.cut()` 函數

使用 `sklearn.preprocessing.LabelEncoder` Transformer

運用機率密度函數(Probability Density Function)

運用機率密度函數(Probability Density Function)，假定數值變數符合常態分佈，因此又稱為 Gaussian Naive Bayes Classifier：

$$P(C_i|x_0, x_1, \dots, x_n) \propto P(x_0, x_1, \dots, x_n|C_i)P(C_i) \propto P(C_i) \prod_{j=1}^n P(x_j|C_i)$$
$$P(x|C_i) = \frac{1}{\sqrt{2\pi\sigma_{C_i}^2}} \exp\left(-\frac{(x - \mu_{C_i})^2}{2\sigma_{C_i}^2}\right)$$

感謝你閱讀完這篇文章，如果你覺得這些文章對你有幫助請在底下幫我拍個手（長按最多可以拍 50 下手）。

上一章：[【Python 機器學習】109：當數據集含有遺漏值的處理方法與未經過訓練的分類預測器](https://medium.com/展開數據人生/python-機器學習-109-當數據集含有遺漏值的處理方法與未經過訓練的分類預測器)